UNIVERSIDADE FEDERAL DO PIAUÍ
Departamento de Computação

Relatório Técnico

Trabalho de apoio computacional ao projeto de tese

Desenvolvimento de modelos de inteligência artificial para predição de sepse tardia em Unidade de Terapia Intensiva Neonatal

Aluno: Caio Victor Ferreira do Nascimento
Professor: Prof. Dr. Vinícius Ponte Machado

Teresina, 31 de outubro de 2026

1 Objetivo

Este relatório resume o que foi desenvolvido até agora no pipeline de aprendizado de máquina do projeto de tese. Ele lista os requisitos que o projeto define, mostra como cada um foi implementado e indica o que ainda falta. As páginas citadas são as do documento do projeto.

O trabalho concentrou-se na parte do pipeline que **não altera os dados**: algoritmos, hiperparâmetros, tratamento do desbalanceamento e validação cruzada. O código está no repositório, no ramo `create-pipeline`, e tem 107 testes automáticos, todos aprovados.

2 Requisitos implementados

Cada requisito abaixo vem da metodologia do projeto. Para cada um, descreve-se como foi implementado e quais arquivos foram criados ou gerados. Os arquivos de código ficam na raiz do repositório. Os resultados de cada execução são gravados na pasta `data/resultados/`, que não vai para o controle de versão porque pode ser regenerada a qualquer momento. A pasta `tests/` tem um arquivo de testes automáticos para cada módulo.

2.1 Oito algoritmos de classificação

O Quadro 4 (p. 44) lista oito algoritmos: Regressão Logística, Naive Bayes, K-vizinhos mais próximos (KNN), Árvore de Decisão, Random Forest, Gradient Boosting, XGBoost e rede neural MLP. Todos foram implementados no arquivo `modelos.py`, que funciona como um catálogo. Para cada algoritmo, ele define o estimador, os parâmetros fixos, como a semente aleatória, e as grades de hiperparâmetros. Sete algoritmos vêm da biblioteca scikit-learn, e o XGBoost vem da biblioteca xgboost.

Cada algoritmo é montado em um *pipeline*, uma sequência de passos em que o pré-processamento vem sempre antes do modelo. Com isso, a cada rodada da validação cruzada, o pré-processamento é refeito só com os dados de treino daquela rodada. O comando `python modelos.py --listar` imprime o catálogo. O arquivo `tests/test_modelos.py` verifica, entre outros pontos, que os oito pipelines treinam e devolvem probabilidades válidas.

2.2 Busca de hiperparâmetros

O projeto prevê a busca de hiperparâmetros com `GridSearchCV` (p. 45). Em `modelos.py`, cada algoritmo tem duas grades. A grade enxuta, com 59 combinações no total, serve para testar o pipeline em poucos minutos. A grade completa, com 325 combinações, está prevista para os dados reais. Na Random Forest, por exemplo, variam o número de árvores, a profundidade máxima, o tamanho mínimo da folha e o número de variáveis por divisão.

O scikit-learn só acusa um valor inválido no momento do treino, o que poderia interromper uma busca longa no meio. Por isso, os testes treinam cada algoritmo com a primeira, a do meio e a última combinação de cada grade e conferem que todo hiperparâmetro existe no estimador. Os valores das grades são iniciais e serão revistos com os dados reais.

2.3 Divisão entre treino e teste

O projeto define 80% dos dados para treino e 20% para teste (p. 45). A divisão é feita pela função `dividir`, no arquivo `preprocessamento.py`, de forma estratificada: as duas partes mantêm a mesma proporção de sepse. Ela gera os arquivos `data/treino.csv` e `data/teste.csv`. Com os dados sintéticos, o treino ficou com 800 pacientes, 160 deles com sepse, e o teste com 200 pacientes, 40 com sepse. O arquivo de teste só é lido na avaliação final.

2.4 Validação cruzada com cinco dobras

O projeto prevê validação cruzada com $k = 5$ (p. 45). Ela foi implementada no arquivo `validacao.py`, com `StratifiedKFold`, que mantém a proporção de sepse em cada dobra. O arquivo produz duas estimativas para cada algoritmo. A primeira é a da própria busca em grade, que é otimista, porque a melhor combinação foi escolhida justamente por ter ido bem naquelas dobras. A segunda é a validação aninhada: em cada uma de cinco rodadas, a busca inteira roda em quatro dobras, e o modelo escolhido é medido na quinta, que ele nunca viu. A comparação entre algoritmos usa a segunda.

Para cada modo de desbalanceamento, o `validacao.py` grava três saídas na pasta de resultados:

- `comparacao_<grade>_<modo>.csv`: uma linha por algoritmo, com as métricas da validação aninhada, a nota da busca simples, os melhores hiperparâmetros e o tempo de execução;
- `oof_<grade>_<modo>.csv`: para cada paciente do treino, a probabilidade de sepse prevista por um modelo que não o viu, usada nas curvas ROC;
- pasta `modelos_<grade>_<modo>/`: um arquivo `.joblib` por algoritmo, com o modelo de melhores hiperparâmetros treinado no treino inteiro.

O arquivo também exige um mínimo de 25 casos de cada classe, porque, com poucos casos de sepse, um F1 igual a zero pareceria um resultado normal. Os testes em `tests/test_validacao.py` mostram, por exemplo, que com o desfecho embaralhado as previsões não acertam mais que o acaso.

2.5 Tratamento do desbalanceamento

O projeto prevê avaliar reamostragem sintética e ajuste de pesos de classe (p. 46; seção 2.2.1, p. 29). O arquivo `modelos.py` oferece três modos, escolhidos na execução do `validacao.py`: sem ajuste, pesos de classe e SMOTE.

No modo de pesos de classe, os erros nos casos de sepse pesam mais no treino. Regressão Logística, Árvore de Decisão e Random Forest têm esse recurso pronto no scikit-learn. O Gradient Boosting não tem, e por isso foi criada a classe `GradientBoostingBalanceado`, que calcula os pesos a cada treino. O XGBoost tem o parâmetro `scale_pos_weight`, mas ele exige um número fixo, calculado antes. Calculado com o treino inteiro, esse número incluiria os pacientes da dobra de validação. A classe `XGBClassifierBalanceado` resolve isso calculando a proporção a cada treino. KNN, Naive Bayes e MLP não aceitam pesos e rodam sem ajuste nesse modo.

O SMOTE, da biblioteca imbalanced-learn, cria casos sintéticos de sepse a partir de casos vizinhos até igualar as classes. Ele cobre os oito algoritmos e é um passo do pipeline aplicado só no treino de cada dobra, nunca na previsão. Um teste mostra por que isso importa: com o desfecho embaralhado, o SMOTE aplicado antes da divisão das dobras infla artificialmente o desempenho, e o SMOTE dentro do pipeline não.

2.6 Métricas de desempenho

O Quadro 5 (p. 45) define F1-score, acurácia, precisão, sensibilidade e AUROC, com o F1 como métrica principal. Todas foram implementadas em `validacao.py`, e acrescentou-se a especificidade, citada no Resumo do projeto. O limiar de decisão é 0,5. O melhor modelo é o de maior F1, e a AUROC desempata. Os testes conferem as métricas com contas feitas à mão.

2.7 Comparação em tabela e curvas ROC

O projeto pede a comparação dos algoritmos em tabela e em curvas ROC (p. 46). O arquivo `avaliacao.py` lê as comparações dos três modos e escolhe o vencedor entre as 24 combinações

de algoritmo e modo, pelo F1 da validação aninhada. Só depois disso ele abre o conjunto de teste. Assim, o teste mede o desempenho, mas não interfere na escolha. Em seguida, mede as 24 combinações no teste e calcula o intervalo de confiança de 95% do vencedor por *bootstrap*, com 1.000 reamostragens. Ele grava:

- `avaliacao_teste_enxuta.csv`: tabela com as métricas das 24 combinações no teste;
- `vencedor_enxuta.json`: o vencedor, suas métricas na validação e no teste, os intervalos de confiança e a matriz de confusão;
- `roc_oof_enxuta_<modo>.png` e `roc_teste_enxuta_<modo>.png`: curvas ROC dos oito algoritmos no treino e no teste, uma figura por modo;
- `confusao_vencedor_enxuta.png`: matriz de confusão do vencedor no teste.

2.8 Explicabilidade com SHAP

O projeto prevê explicar o melhor modelo com valores SHAP (p. 46). O arquivo `explicacao.py` decompõe a probabilidade prevista para cada paciente na contribuição de cada variável, com o mesmo método para qualquer algoritmo vencedor. Ele grava:

- `importancia_shap_enxuta.csv`: a contribuição média de cada variável, da maior para a menor;
- `shap_resumo_enxuta.png`: gráfico das 15 variáveis mais importantes;
- `shap_caso_enxuta.png`: explicação de um alerta individual, mostrando quais variáveis levaram o modelo a indicar sepse;
- `modelo_final.joblib`: o modelo vencedor completo, da tabela bruta até a probabilidade;
- `modelo_final.json`: descrição do modelo final, com algoritmo, hiperparâmetros, forma de escolha, colunas esperadas, limiar e métricas.

O arquivo também tem a função `prever`, que recebe uma tabela de pacientes novos, confere as colunas e devolve a probabilidade de sepse de cada um. Os testes em `tests/test_explicacao.py` verificam que as contribuições somam exatamente a probabilidade prevista.

3 Por que os dados ainda não foram trabalhados

Os dados reais ainda não existem. A coleta depende da aprovação nos Comitês de Ética e do acesso à API do prontuário SOUL MV, com coleta prevista para 2027 (p. 48). Além disso, há dúvidas no projeto, listadas na seção 4, que mudam quais dados devem ser coletados e como.

Por isso, o pipeline foi testado com **dados sintéticos**: 1.000 pacientes fictícios com as variáveis dos Quadros 2 e 3 e 20% de sepse. Para os algoritmos rodarem, foi feito um pré-processamento mínimo e provisório, com preenchimento de valores ausentes, padronização e codificação de categorias. Os resultados obtidos servem só para mostrar que o pipeline funciona e não têm valor científico.

4 Dúvidas sobre o projeto

1. **Qual pergunta o modelo responde.** A introdução fala em prever a sepse com antecedência (p. 13), entre todos os internados. Já os exames são colhidos no momento da suspeita de sepse (p. 39 e 40). Para quem nunca teve suspeita, não está definido qual exame usar. Se casos e controles tiverem exames de momentos diferentes, o modelo pode aprender a reconhecer quem estava sendo investigado, e não a sepse.

2. **Variáveis que podem ser consequência da sepse.** As variáveis são colhidas na "última evolução do paciente" (p. 38), que é posterior ao diagnóstico. Totais de dias de UTIN, de ventilação e de cateter, o resultado da hemocultura e o "tempo até o diagnóstico" podem entregar o desfecho ao modelo.
3. **Excluir ou imputar dados ausentes.** A p. 38 manda excluir prontuários incompletos, e a p. 40 manda imputar os valores ausentes.
4. **Escores clínicos.** qSOFA e NEWS2 usam pontos de corte de adultos, que quase todo recém-nascido atinge (p. 33). O nSOFA, feito para neonatos, é citado no texto, mas não está na ficha de coleta.
5. **Critério de sucesso.** O projeto não define o que é "desempenho adequado" (p. 14) nem o limiar de decisão do modelo.

5 O que falta implementar

- Extração dos dados do SOUL MV pela API e cálculo dos escores clínicos (p. 43).
- Pré-processamento definitivo, que depende dos dados reais e das dúvidas acima.
- SVM e redes profundas, citados no Quadro 4, com a regra de exclusão abaixo de 70%.
- Análise das variáveis associadas à sepse (objetivo específico 1).
- Protótipo do sistema de apoio à decisão (objetivo específico 3). Já existe a base: o melhor modelo é salvo e tem uma função que devolve a probabilidade de sepse de novos pacientes.

6 Alterações quando os dados chegarem

- Trocar o gerador de dados sintéticos pela extração real.
- Atualizar os nomes e grupos de colunas, concentrados no início de `preprocessamento.py`.
- Remover ou recortar no tempo as variáveis que dependerem das respostas às dúvidas 1 e 2.
- Refazer o pré-processamento de acordo com os dados reais.
- Rodar a grade completa de hiperparâmetros e revisar seus valores.

Os algoritmos, a validação cruzada, o tratamento do desbalanceamento, a avaliação e o SHAP não precisam mudar.